UID:

1a./1b.

1a) Base case: $n = 1$ $\quad T(1) = aT(1/b) + c(1)^d$

$T(1) = a^{\log_b(1)} T(1) + c(1)^d \left(\sum_{j=0}^{-1} (a/b^d)^j \right)$ $\quad \not= aT(1/b) + c$

$T(1) + c\underbrace{\left(\sum_{j=0}^{-1} (a/b^d)^j \right)}_{0}$

$T(1) = T(1)$ ✓

IH: Assume lemma 0.3 holds for all n ($n = b^k$ for some $k \geq 0$)

IS: $T(b^{k+1}) = aT(b^{k+1}/b^k) + c(b^{k+1})^d$

$= aT(b^k) + c \cdot (b^{k+1})^d$

$= a\left[a^{\log_b b^k} T(1) + cb^{kd} \left(\sum_{j=0}^{\log_b b^k - 1} (a/b^d)^j \right)\right] + c(b^{k+1})^d$

$= a^{k+1} T(1) + acb^{kd} \cdot \sum_{j=0}^{k-1} (a/b^d)^j + c\, b^{(k+1)d}$

$= a^{k+1} T(1) + \sum_{j=0}^{k} (a/b^d)^j + cb^{(k+1)d}$

reorder $= a^{k+1} T(1) + cb^{(k+1)d} + \sum_{j=0}^{k} (a/b^d)^j$ □

b) ① $\frac{a}{b^d} = 1$

$T(n) = a^{\log_b(n)} T(1) + cn^d \left(\sum_{j=0}^{\log_b n - 1} (1)^j \right) = a^{\log_b(n)} T(1) + cn^d \cdot \log_b(n)$

constants ↓ ↓

lemma 0.1 ↘

$= n^{\log_b a} + n^d \log_b(n)$

$= n^d (\log_b n)$ □

$\log_b n$ is goes to 0

② $\frac{a}{b^d} < 1$ $\quad \sum_{j=0}^{\log_b(n) - 1} \left(\frac{a}{b^d}\right)^j = \frac{1 - \left(\frac{a}{b^d}\right)^{\log_b(n)}}{1 - \frac{a}{b^d}} \simeq 1$

$T(n) = a^{\log_b n} T(1) \cdot cn^d \cdot 1$

$= n^{\log_b a} \cdot n^d$

$\log_b a < d$, so n^d dominates

$= O(n^d)$ □

1b. ③ $d < \log_b a$ $\quad a/b^d > 1$

similar process to last one

$$\sum_{j=0}^{\log_b n - 1} \left(\frac{a}{b^d}\right)^j = \frac{1 - \left(\frac{a}{b^d}\right)^{\log_b n}}{1 - \frac{a}{b^d}} \longrightarrow \text{dominates}$$

by lemma 0.1 $\quad n^{\log_b (a/b^d)} = n^{\log_b a - d}$

$T(n) = a^{\log_b n} T(1) \cdot cn^d \cdot n^{\log_b a - d}$

$= n^{\log_b a} \cdot n^{\log_b a}$

$= n^{\log_b a}$ □

2.

Algorithm:

Idea: Perform binary search on the array, A to find the smallest element in the array.
Subproblems: At each step divide the array into two halves.

1. Initialize int left = 0, int right = len(A) - 1.
2. While (left < right)
    a. int mid = (left + right) // 2
    b. If A[mid] > A[right]
        i. left = mid + 1
    c. Else
        i. right = mid
3. Return len(A) - left

Proof:

Correctness:

Prove an inversion exists. Prove algorithm finds inversion.

Claim: An inversion exists in a shifted array.

By definition, an inversion exists if i > j and A[i] < A[j]. Say in the original sorted array, i is the smallest element and j is the largest element meaning i < j and A[i] < A[j], by definition of an ascending order array. We shift the array left by k spots. Now, i > j but the element A[i] < A[j]. This is an inversion pair, and the largest element is now followed by the smallest element. [i…j] to [...j,i….]. Therefore, we prove our claim that if our array is shifted, an inversion pair must exist because the shift introduces this change in the pair's relationship.

Since the original array was sorted, we know that the smaller element in the inversion is where the shift started from. The smallest number that follows the largest number now should actually be at position 0, thinking back to our sorted array. Therefore, subtracting the smallest element's new position from the length of the total array will tell us how much it has been shifted to the left from its original 0 spot.

Claim: My algorithm finds the inversion.

My approach was binary search with some modifications. Instead of finding a target, we are finding the smallest element, which is part of the inversion pair. We keep calculating a midpoint of the current array subset we are working with and then eliminate half of the array until the left and right points are equal (i.e. until we have searched the whole array) and there is only one element left to search. If the midpoint is greater than the right, the smallest element that started the shift must be in the right half since the original array was sorted–so we search that half. If the midpoint is smaller than or equal to the right, the smallest element cannot be in the right half (since it is in increasing order) so we search the left half. Once the left and right are equal, we found the index of the smallest element (left). Based on the explanation above this claim, as well as my proof of k underneath this paragraph, the way that my algorithm uses the smallest

element finds k, the number of spots the array shifted. Therefore, my algorithm finds the inversion.

Proof of calculating k:

- Base case: Suppose we have an array where n = 1. len(n) - left = 1-0 = 1. We have a shift of 1.
- Inductive hypothesis: Assume this algorithm is correct in calculating k and that for n = z, len(z) - left = length of shift = k.
- Say the z + 1 element is smaller than the original smallest element, and the shift number is still the same. Now, the new smallest element is moved to the right by one spot because of one extra element. The length is also extended by 1. This results in (len(z)+1) - (left+1) = len(z) - left = same shift length.
- Let the z + 1 element be larger than the smallest element, and the shift number is the same. The length is again increased by 1. Also, the smallest element is moved over one to the right. (len(z) + 1) - (left + 1) = k.

Edge Case: No shift/shift k = n
If we shift an array by n, then the smallest element ends up at 0 again. n - 0 = n, so we correctly account for this edge case.

Completeness:
The condition of the while loop ensures that we look through the whole array. Therefore, we are guaranteed to find the smallest element and calculate k correctly. The algorithm is also O(logn), fulfilling the problem's requirements.

Termination:
Once left and right equal one another, the while loop will end since left = right, and we have searched the whole array. Therefore, the algorithm will terminate.

Time Complexity: O(logn)

- Step 1, 3, and everything in the body of the while loop are constant time. O(1)
- Iterations of while loop:
    - My approach was binary search with some modifications. At each step, you are dividing the array into halves. Since binary search has a time complexity of O(logn), we can apply that here too.

3.
Algorithm:
Array A and B.

1. lowA = 0
2. lowB = 0
3. highA = len(A) - 1
4. highB = len(B) - 1
5. While (lowA <= highA and lowB <= highB)
    a. midA = (lowA + highA) // 2
    b. midB = (lowB + highB) // 2
    c. If (midA+midB+1 < k)
        i. If A[midA] < B[midB]
            1. lowA = midA + 1
        ii. Else
            1. lowB = midB + 1
    d. If (midA+midB+1) >= k)
        i. If A[midA] > B[midB]
            1. highA = midA - 1
        ii. Else
            1. highB = midB - 1
6. If (lowA <= highA) // only A is left
    a. Return A[lowA]
7. If (lowB <= highB) // only B is left
    a. Return B[lowB]
8. Return min(A[lowA], B[lowB])

Proof:
Correctness:
I used a version of binary search in my solution, where I did binary search on A and B separately. But for shrinking the subset of the arrays we are looking at, I compared their midpoints to one another.

Proof by invariants: Claim: The kth smallest element is always in the subsets you are looking at (between low and high of the arrays). I will prove that whatever we eliminate is NOT the kth smallest element.

At the beginning, low and high encompasses the full range of both arrays. The kth smallest element has to come from somewhere in one of the arrays, so if we are looking at all the elements we have to have the kth smallest element in our search space.

In each iteration of the binary search, we adjust the space we are searching by shifting either the low or high of one of the arrays. We do this by comparing the mid elements.

If the sum of all the elements we are looking at total is less than k, we have not yet reached the kth smallest element since we do not even have k elements. Therefore, we need to move forward in the array that has the smaller mid element. We remove fewer than k smallest elements from the search space, ensuring that the kth smallest element is still in the search space, in one of the remaining 3 halves.

If the sum of all elements we are looking at is more than k or is k, we have either passed the kth smallest element with our mid element or we are at it. So, we shrink our array by eliminating the upper half of the array with the larger mid element. This larger mid element is greater than everything in the left half of its array as well as the left of the other array (since if it is bigger than the other array's mid value it is bigger than everything below it too. The kth smallest value cannot be in the upper half of this array because by my previous sentence, there are at least k elements smaller than it, so it has to be the k+1th smallest element or more.

The elements we are considering are consistently reducing the search space, and in these reductions we are ensuring that the kth smallest element remains in our search space.

Once low is greater than high in an array, we have eliminated that whole array from consideration. By our previous rules, this means that the kth smallest element is found in the portion from low to high of the other array that we have not eliminated yet. Even at the end, again the kth smallest element remains in our search space.

Completeness: We find the kth smallest element, as shown by the correctness proof. Also, we keep decreasing the search space until one array is exhausted, which ensures we will find the kth smallest element in the other array. The algorithm's time is faster than linear time, as discussed below, so we fulfill the requirement as well.

Termination: We only want to end the while loop when one array's search space is essentially empty, because we have looked through the whole thing and know the kth smallest element is not there. When we have looked at the whole array its low value crosses over, or is larger than, its high value. This would invalidate the while loop, so we know the algorithm terminates. We know the algorithm has to end since the kth smallest element has to exist somewhere, so once we figure out which array it is in, our algorithm will end.

Time Complexity: O(logn)

- Going off of binary search time complexity, we are splitting two n-sized arrays in halves until we are down to no elements in one. This overall is 2*O(logn) time, which simplifies to O(logn).
- Everything not in the while loop is O(1) time.

4.

Algorithm:

1. If n = 1, then there is only one bank card that corresponds to one account and it is the majority
    a. Return that card
2. Split the cards in half. Let the left half be L and right half be R.
3. majorityL = Call algorithm on L.
4. majorityR = Call algorithm on R.
5. Use tester to see if majority element card's on L's account and R is same
6. If majorityL == majorityR
    a. That's the overall majority, return majorityL
7. Else
    a. Count number of majorityL in entire set by testing it with the whole set, call that countL (if majorityL exists)
    b. Count number of majorityR in entire set by testing it with the whole set, call that countR (if majorityR exists)
    c. If countL is > n/2
        i. Return majorityL
    d. If countR is > n/2
        i. Return majorityR
8. Return false - there is no majority element in the set

If a card is returned, we have a majority element. If false is returned, no majority element in the set.

Proof: Proof by Invariants.

Claim: At any point in our algorithm, the majority element of the set, if there is one, must be the majority of either the current right, the current left half, or both if they are the same.

Base case: When we have just one card, that card is trivially the majority since there are n cards with that majority, which is > n/2 by construction. So, that is what we return. Therefore, we know the majority element and the invariant holds.

In the recursive calls, we either have majority elements that are equal on both sides, different ones on both sides (which also includes one being false and one having a majority element), or no majority element on either side so no majority element overall.

If the majorities of both halves are equal, the invariant holds. By definition, a majority element means more than half the cards in that subset are equivalent accounts. If majorityL and majorityR are equal, this means the element is the majority in both halves and has to make up more than half of the entire set too (a majority in each half ensures when you add the total count together it is the majority overall). The invariant holds because the set's majority element was the majority element of one of the halves.

If both halves have a majority element but they are different, they are the only contenders for the overall majority because in order to have a majority element of the overall set, it would have to be a majority element of at least half of the set. If we have a majority that matches more than n/2 elements, any other element can match at most *less* than n/2 elements, since there are only that many elements left. This rules out any other contenders, the invariant holds and the majority element of the set is the majority of the left or right half. We would proceed by cross-checking each majority across the whole set to find which one is more than n/2 elements. Again, it can only be one or neither of them since we have n elements total and if one is n/2 + 1 elements, that leaves less than n/2 possible for the other.

If neither half has a majority element, then we return false which is correct because the overall set cannot have a majority element unless it is the majority of a half. No element in the left half occurs more than |L|/2 times, and no element in R occurs more than |R|/2 times. The maximum occurrences of any element is |L|/2 + |R|/2 = n/2 at most. This is *not* more than n/2 so we cannot have a majority element in the overall set unless it is the majority of a half.

Overall, the invariant holds throughout our algorithm so we correctly find the majority element, if it exists. At any given point in the algorithm, the overall majority must be the majority of at least one of the halves. If none exists in the halves, then there is no overall majority.

Completeness: We search the entirety of the set through recursively splitting the set into halves until we get to 1 card and then propagating up. The only operation we do directly with the cards is picking two and testing two at a time, as required by the problem. The time complexity also matches the requirement, as discussed below. Finally, we overall if the majority element exists, we return a card with it and if not, we return false.

Termination: There are no loops. In terms of recursion, we cannot have an infinite recursion since the smallest recursion in terms of splitting the arrays in half is getting to a size of 1. And when we do, we return something - ending that recursive iteration. Once we have combined all the halves together, we return a majority or false. All the other steps are finite.

Time Complexity Analysis: O(nlogn)

- Step 1 and 2 are O(1).
- Since we are splitting in halves, we make recursive calls on two n/2 size arrays at a time.
    - The recurrence for this is T(n) = 2T(n/2) + combination step.
- Combination step: When the two halves do not have the same majority, we iterate over n elements when comparing an array's majority card to all the cards in the other array. This takes O(n) time. If the halves have the same majority, the combine step is just O(1).
- Overall, we take the worst case combination to get T(n) = 2T(n/2) + O(n). This simplifies to O(nlogn), as we showed in class.

5. Algorithm:

1. Sort the lines in order of increasing slope.
2. Initialize a set S.
3. If n = 1,
    a. Add the line to S. It is trivially visible.
    b. Return S.
4. If n = 2,
    a. Add both lines to S. Since we have distinct slopes, before the intersection point one is higher and after that, the other is higher.
    b. Return S.
5. If n = 3,
    a. Add the first and last line to S.
    b. Calculate the intersection point of each pair of lines.
    c. If (the second line intersects with the first line below where the first and third lines meet)
        i. Add the second line to S.
        ii. Return S.
    d. Else
        i. Return S.
6. Split the set of lines in half. Let |L| = ceiling(n/2) and |R| = floor(n/2)
7. setL = Algorithm(L). set R = Algorithm(R).
    a. Add the first line of setL and the last line in setR to S, they are always visible.
    b. Loop through elements in setL and setR until you reach the end of either set..
        i. Calculate where two lines intersect.
        ii. Whichever one is higher BEFORE the intersection point, add that to S.
        iii. Move forward in whichever set you just removed the line from and keep doing comparisons.
8. Return S.

Proof:

Correctness:

Proof by induction:

Base Case: n <= 2. When we have one element, no lines can be hidden so the algorithm is correct. When we have two elements, no element can be hidden again because by definition of distinct slopes, there must be a point where each is higher than the other. The algorithm is correct.

Inductive Hypothesis: The algorithm is correct for a set of k lines, and only returns whichever lines are visible at any point.

Inductive Step: For the k+1th line, if it is the smallest slope or largest slope, it is added to the set since it is visible at a point, as explained in the base case. If not, we need to find the lines to the left and right of it (by slope). If the k+1 line is highest at a point before it intersects with the right line, (which by construction is higher than all the other lines so far), then it should be added to

the set. If not, then it would not be the highest point at any point, since after the intersection point the right line will always be higher than the k+1 line. If we add the k+1 line to the set, it is higher than the right line at some region, so it also must be higher than all the lines before in that region too. By this approach, we only add lines if they are the highest at some point on the graph, and therefore visible.

Proof for sorting strategy: If we sort lines in ascending order by slope, we ensure that the intersection points of consecutive lines occur from left to right as the x-coordinate increases. This is why when we are comparing two lines, we can just compare the two next to each other, because if a line is higher than the one with the slope right below it, it has to be higher than all the ones before that too.

Completeness: We meet the time complexity requirement, as shown below. As shown by the correctness proof, we only add lines to the overall set if they are the highest point for some x. This meets the visibility requirement.

Termination: The recursion ends when we get to a base case of 1 or 2 lines, so we do not have an infinite recursion. The loop comparing the elements in setL and setR ends when we go through all the elements in both sets, so there is no infinite loop. Thus, the algorithm terminates.

Time Complexity Analysis: O(nlogn)

- Sorting the lines by slope takes O(nlogn) time.
- Steps 2-5 are constant time O(1).
- Splitting the set is O(1).
- The recursion is 2T(n/2).
    - The combination step is n comparisons, so it is O(n).
    - Overall this 2T(n/2) + O(n) which simplifies to O(nlogn) as we showed in class.
- 2*O(nlogn) is just O(nlogn).